CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 7

Name: Prionti Das
UID: 24BAI71005
Branch: CSE(AIML)
Section/Group: 24AIT_NTPP-3[B]
Subject: Full Stack - II
Date of Performance: 24/09/2026
Subject Code: 24CSP337
Submitted to: Ms. Lovleen Kaur (E20454)

EXPERIMENT – 7.1

1. **AIM:** To implement secure backend APIs using JWT authentication and role-based authorization in Spring Boot with Spring Security.

2. **OBJECTIVE:**

   1. To understand JWT-based authentication.
   2. To configure Spring Security for API protection.
   3. To implement JWT authentication filters.
   4. To enforce role-based access control (RBAC) using Spring Security.
   5. To restrict API access according to user roles such as ADMIN and USER.

3. **SOFTWARE REQUIREMENTS:**
   1. Java Development Kit
   2. Spring Boot
   3. Spring Security
   4. Maven
   5. Visual Studio Code

4. **THEORY:**

   JWT (JSON Web Token) is used for secure, stateless authentication in backend applications. After successful login, the server generates a token that the client includes in subsequent API requests. Spring Security validates the token through a security filter chain and protects restricted endpoints. Role-Based Access Control (RBAC) assigns permissions according to user roles, such as ADMIN and USER. The `@PreAuthorize` annotation provides method-level security by restricting access to specific APIs based on user roles. Together, JWT and Spring Security help ensure secure authentication and authorization in Spring Boot applications.

5. **IMPLEMENTATION/PROCEDURE:**
   1. Create a Spring Boot project with Spring Security and the required dependencies.
   2. Configure Spring Security to protect REST API endpoints.
   3. Implement JWT generation and validation for user authentication.
   4. Create a JWT authentication filter to validate tokens in incoming requests.
   5. Define user roles such as ADMIN and USER.

24BAI71005_Prionti

6. Apply role-based restrictions using `@PreAuthorize`.
7. Test authentication and authorization using Postman.

6. CODE :

```
async function authenticate(event) {
 event.preventDefault();
 setLoading(true);
 setMessage(null);

 try {
  const endpoint =
   mode === "login" ? "/auth/login" : "/auth/register";

  const response = await fetch(`API{endpoint}`, {
   method: "POST",
   headers: {
    "Content-Type": "application/json",
   },
   body: JSON.stringify({ username, password }),
  });

  const data = await response.json();

  if (!response.ok) {
   throw new Error(data.message || data.error || "Authentication failed");
  }

  localStorage.setItem("security_token", data.token);
  localStorage.setItem(
   "security_user",
   JSON.stringify({
    username: data.username,
    role: data.role,
   })
  );

  setToken(data.token);
  setUser({ username: data.username, role: data.role });
 } catch (error) {
  setMessage({
   type: "error",
   text: error.message || "Unable to connect to the backend.",
  });
 } finally {
  setLoading(false);
 }
}
```

24BAI71005_Prionti

7. OUTPUT:

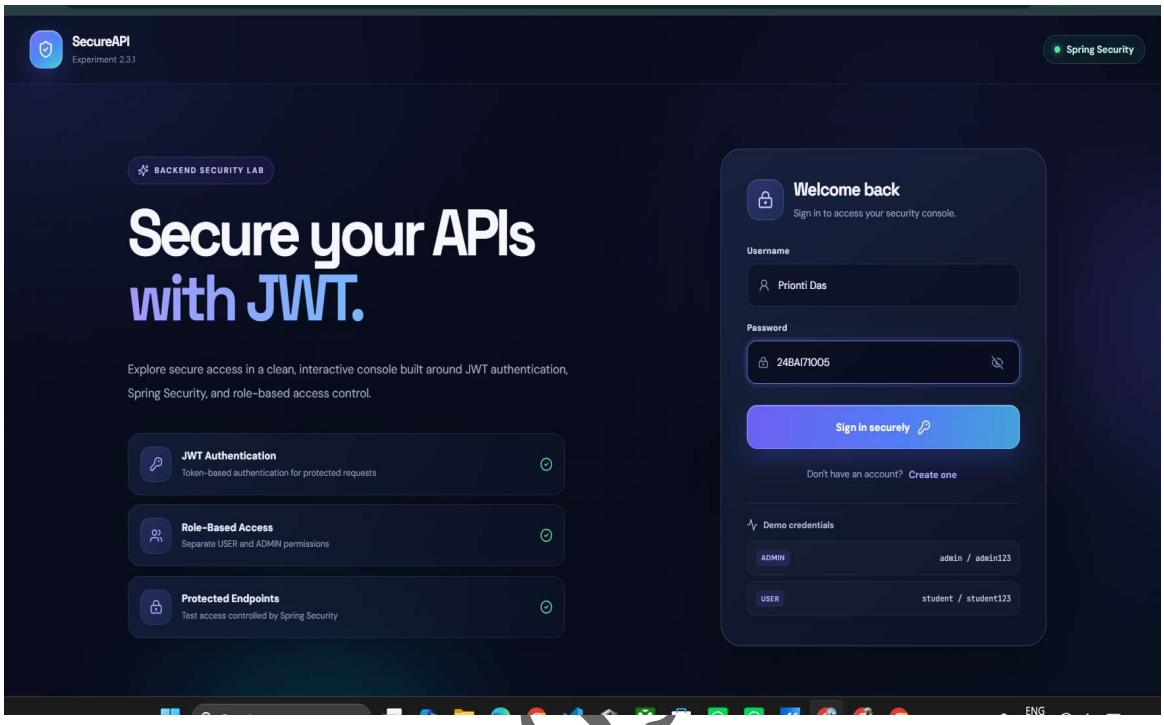


Fig 7.1: User Interface of JWT Authentication and Role-Based Access Control (RBAC) System

The designed interface provides a modern and user-friendly UI for JWT authentication and role-based access control. It includes login and registration forms, a secure dashboard, and protected API endpoints for USER and ADMIN roles. Successful operation requires the Spring Boot backend to be running.

8. LEARNING OUTCOME:

- Learned JWT-based authentication in Spring Boot.
- Configured Spring Security to protect REST APIs.
- Implemented JWT authentication and token validation.
- Understood role-based access control (RBAC).
- Applied role-based restrictions to secure API endpoints.

24BAI71005

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 7.2

1. **AIM:** To implement encryption and secure storage mechanisms for protecting sensitive data and credentials in a Spring Boot application.

2. **OBJECTIVE:**

   1. To understand encryption techniques for protecting sensitive data.
   2. To implement AES-based encryption for securing sensitive data.
   3. To securely store third-party credentials using encryption mechanisms.
   4. To manage token lifecycle using access and refresh tokens for secure authentication.

3. **SOFTWARE REQUIREMENTS:**
   1. Java Development Kit
   2. Spring Boot
   3. Maven
   4. Visual Studio Code

4. **THEORY :**

Encryption is a security technique used to protect sensitive data by converting readable information into an unreadable format. AES (Advanced Encryption Standard) is a symmetric encryption algorithm that uses the same secret key for encryption and decryption.

In Spring Boot applications, sensitive information such as OAuth tokens and third-party credentials should be encrypted before being stored in a database. Access tokens are short-lived and used to access APIs, while refresh tokens are used to obtain new access tokens. Token expiration and rotation help reduce security risks and improve session management.

These mechanisms ensure secure credential storage, data confidentiality, and improved backend security.

5. **IMPLEMENTATION/PROCEDURE:**
   1. Create a Spring Boot project with the required dependencies.
   2. Implement AES encryption and decryption using Java Cryptography libraries.
   3. Encrypt sensitive credentials before storing them in the database.
   4. Implement access and refresh token generation and validation.
   5. Configure token expiration and refresh token rotation.
   6. Test encryption, secure storage, and token management.

6. **CODE :**

```
<div className="demo-box">
  <div>
    <UserRound size={15} />
    <span>Demo account</span>
  </div>

  <strong>Prionti / 24BAi71005</strong>
```

24BAI71005_Prionti

```
<small>
  Use only if this account exists in your local backend.
 </small>
</div>
```

7. OUTPUT:

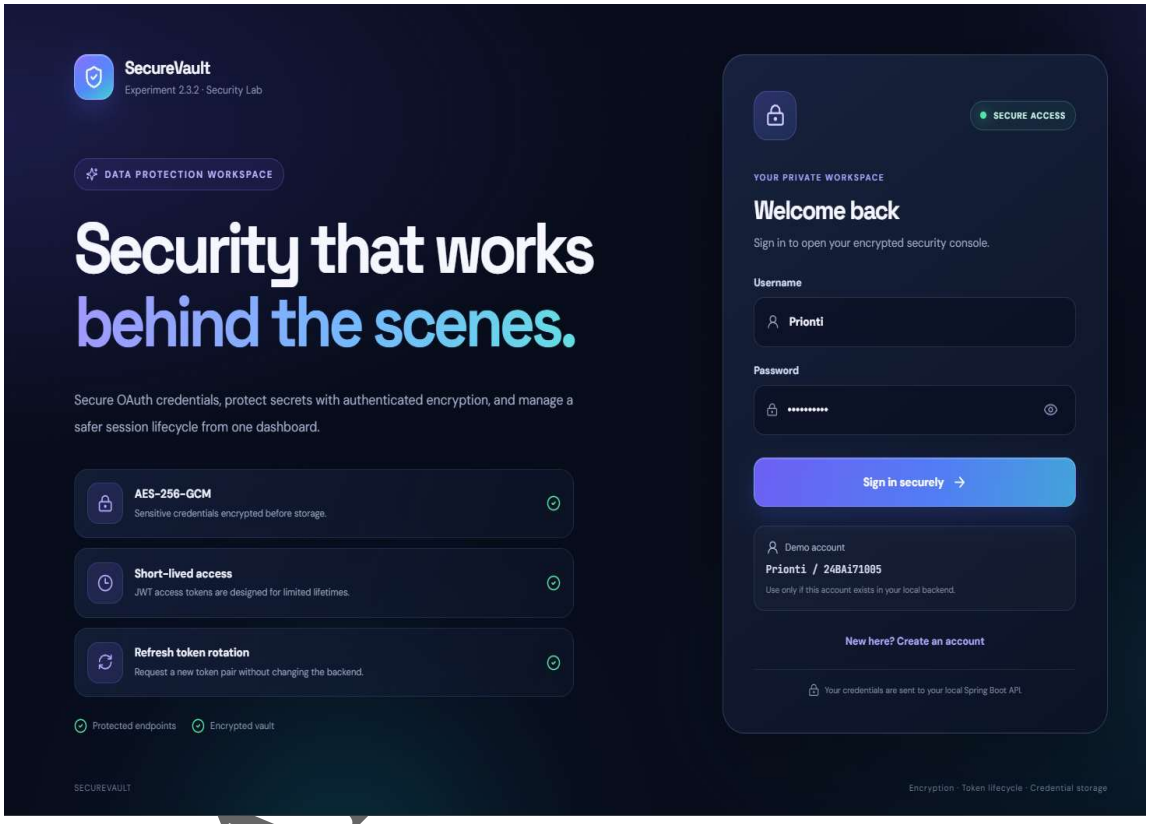


Fig 7.2:SecureVault User Interface for AES Encryption, Secure Credential Storage, and JWT Access–Refresh Token Management.

The SecureVault application displays an attractive and user-friendly interface for secure credential management. It provides login and registration options, AES-256-GCM encryption for sensitive credentials and JWT access and refresh token management. The dashboard allows users to store credentials securely and test protected API endpoints.

8. LEARNING OUTCOME:

- Understood encryption techniques for protecting sensitive data.
- Implemented AES-based encryption and decryption.
- Learned secure storage of sensitive credentials.
- Understood access and refresh token management.
- Learned token expiration and rotation for enhanced security.